DPH User Guide

Delphi Project Handoff

Windows desktop experimental software

Last changed 12 October 2026

For the person inspecting a Delphi project and recording its handoff knowledge

This guide describes the implemented application and its evidence limits. Use the phase acceptance records for exact measured results and unresolved release gates.

Contents

Purpose and evidence....4
Start DPH....4
Run the application and its checks....4
Attach a source project....5
Open cached work....5
Manage the project catalog....6
Record the selected application profile....6
Define and review business terms....6
Learn the selected project....7
Measure coverage before relying on an explanation....9
Explore and search....9
Search across the captured project....9
Read source and graph evidence....9
Investigate source quality candidates....10
Compare captured revisions....10
Choose graph scope and grouping....11
Read a routine as a beginner....11
Inspect screens....11
Inspect keyboard shortcuts and focus order....12
Inspect menus context menus and toolbars....12
Inspect dynamic and runtime created controls....13
Inspect responsive layouts anchors and alignments....13
Inspect component dependencies and support records....14
Inspect inherited forms and embedded frames....14
Import actual screenshot evidence....15
Inspect database connections and datasets....15
Inspect database engines and versions....16
Inspect tables views and schemas....16

Inspect column types defaults and constraints....17
Inspect primary foreign and unique keys....18
Inspect indexes and performance evidence....18
Inspect stored procedures functions and triggers....19
Inspect entity relationship diagrams....20
Define table and column business meanings....21
Follow dataset and screen field bindings....22
Inspect database read and write operations....22
Inspect transaction and rollback paths....23
Inspect field validation and transformations....24
Trace inputs and destination candidates....24
Inspect migration and schema-version history....25
Inspect import and export file formats....26
Record target data retention and archival policies....26
Record target data quality rules and check reports....27
Compare screen history and visual changes....28
Record user actions and business purposes....28
Write attributed knowledge....29
Link authored sections to exact evidence....29
Review and retire knowledge....30
Review changed knowledge against evidence....30
Answer focused author questions....30
Preserve saved runtime observations....31
Retain captures and review rename continuity....31
Back up and restore a complete database....31
Publish manuals and handoffs....32
Translate and review saved sections....32
Publish localized text and source browsing....33
Validate a bundle in the application....33
Validate and preserve exports....33
Troubleshoot common states....34
Practice a maintenance handoff....34
Understand implementation status....34
Source and revision evidence....35

Purpose and evidence

Delphi Project Handoff explains an explicitly selected Delphi project through source inventory, form information, symbols, relationships, control flow and attributed knowledge. It stores its work in one SQLite knowledge database per selected project. DPH is experimental Windows desktop software. Its static analysis is not a Delphi compiler and does not prove every runtime behavior.

Use the generation number, source hashes, certainty labels and diagnostics together. A complete captured analysis means the implemented analysis stages accepted their captured inputs. It does not mean a human approved the business explanation. PARTIAL means unavailable dependencies, unsupported syntax or bounded analysis prevented full coverage. DPH does not compile or execute the selected project, run its build scripts, load its component classes or connect to its production services.

Start DPH

Start DelphiProjectHandoff.exe from its deployed folder. Keep the migrations folder and Microsoft WebView2 loader with the deployment. WebView2 Runtime supports the optional embedded HTML preview. An unavailable preview must leave the exported HTML usable independently. The initial validated target is Windows Win32; another operating system has not been validated.

The Getting started page opens before a project is loaded. It explains the first ten minutes, each workspace tab, a small learning exercise, Delphi terms and repeatable checks. The header Getting started button returns to it at any time. Its native text fallback remains available without WebView2. Keep the help folder with the deployment.

DPH opens maximized to fill the available Windows desktop. The far-left list contains project objects, the centre contains workspace tabs, and the right-hand pane is labeled Selection details. Click an object in the far-left list to read its identity, signature and source location on the right. Its text wraps to fit; scroll inside the pane for more. Choose Explain in the upper-right page selector for behavior or Source for code. Drag either vertical divider to resize the panes. In a smaller restored window, scroll horizontally below the top command buttons to reach commands that do not fit.

Read current activity and captured status on the line along the bottom edge of the window. To read analysis gaps, choose Learn the project in the upper-right page selector, then click Coverage and troubleshooting in that tab's topic list. PARTIAL means incomplete evidence. The last successfully opened database identity is remembered in the local user profile; reopening shows cached facts before source availability is checked.

Run the application and its checks

To run the current source build, open C:\New Delphi Projects\Delphi Project Handoff\bin\Win32\Debug in File Explorer. Double-click DelphiProjectHandoff.exe. Keep migrations and help in the source checkout root, and WebView2Loader.dll beside the executable. A development launch locates its source checkout; an extracted distribution keeps all three beside the executable. Select a different Delphi application to analyze; DPH rejects itself.

For a first check, choose the intended Debug or Release and Win32 or Win64 analysis configuration, then Attach project. Save its database outside that selected source. Wait for the activity line to finish, select a recognizable form event, and follow its handler through Explain, Source and Graphs. Read Coverage before drawing conclusions. Close and reopen its saved database and confirm the same captured identity and notes.

For developer regression checks, open an elevated Command Prompt, run cd /d "C:\New Delphi Projects\Delphi Project Handoff", then run tools\validate-completion-wp1.cmd. A nonzero exit or FAIL is a failure. Run tools\build-learning.cmd for source lessons and real WebView2 checks, tools\build-wp1.cmd for storage and migrations, and tools\build-wp6.cmd for publishing. These tests create disposable fixtures, not user projects.

Automated checks cannot substitute for an independent beginner explaining an actual program. Record the participant, project, chosen event, identified branches and effects, unresolved questions and outcome. Do not mark the first-time-user acceptance gate passed without that evidence.

Attach a source project

Click Attach project at the top and select the application you want to document using its dproj, dpr or groupproj file. DPH rejects its own project, including renamed copies retaining its GUID or program declaration. The current workspace stays loaded when a selection is rejected. Choose a database filename ending in dphdb. The save dialog defaults to Databases under the Handoff application folder: the top-level Databases folder in the source checkout, or beside the executable in an extracted distribution. Existing databases are not moved. You may browse elsewhere outside the selected source folder. DPH rejects database and export locations inside the source boundary and rejects reparse paths that could bypass that boundary. Keep each selected project in its own database.

Select Debug or Release and Win32 or Win64 as appropriate before analysis. Win64 is an analysis configuration choice; the current DPH executable is built for Win32. Extra defines use semicolons. Explicit dependency roots also use semicolons. Add only roots you intend to permit DPH to read. A missing dependency is a visible gap, not a reason to let discovery search neighboring projects without a boundary.

For the pinned VCL2FMXConverter Win32 reference, enter MSWINDOWS;WIN32 as extra defines to reproduce the recorded baseline. DPH does not synthesize every compiler built-in from the platform label. Supply the symbols required by the selected compiler environment; unknown conditional expressions remain uncertain.

Analysis runs in the background. Explorer data belongs to its displayed generation while the worker prepares another. Changed inputs during a scan supersede the capture and request reassessment. A partial or failed capture does not replace a previous complete published head. DPH retains the evidence needed to inspect partial results honestly.

Open cached work

Click Open cache at the top to inspect a saved dphdb database. The picker defaults to Databases under the Handoff application folder; browse to your existing database if it is elsewhere. DPH validates the cached workspace artifact against its SHA256 and generation before displaying it. Cached identity facts also reject old self-analysis databases without relying on their filename or the original source being present. A rejected cache leaves the current workspace intact. A rejected remembered or command-line startup cache leaves Getting started visible. Cached facts are displayed before source verification. When the captured entry is available, DPH reconnects and verifies in the background; otherwise the cache remains detached.

Detached mode allows historical inspection and authored notes. It does not follow cached filesystem paths to display source text or approve a current-source review. Attach the source when you want live hash checks,

source navigation and human verification against current evidence. A missing or damaged cached artifact is reported rather than silently treated as a current view.

A damaged supplemental screenshot does not block opening valid cached analysis. DPH reports Screenshot evidence unavailable and omits the rejected image from display. Repair or restore its managed evidence before publishing a bundle that uses it; do not replace its hash to disguise damaged content.

Keep a copied database with its adjacent database.dphdb.views folder. DPH resolves cache files beside the opened database and verifies their hashes instead of following old absolute artifact paths. To move the source project, choose Relocate verified source and select the same project entry. Exact file identities, content hashes, availability and analysis selection must match the captured generation. A different copy is rejected. Accepted relocation preserves logical symbols and notes, requests human review and reassesses the new context.

Manage the project catalog

Choose Projects in the page selector at the upper right. Refresh the catalog, select the intended database and open it. Missing entries identify unavailable paths; they do not mean the project or its authored history was deleted. Remove an entry only when you want to remove that catalog reference. Catalog removal does not delete the database or managed evidence.

Use one database per analyzed application. Keep its adjacent .views directory with it. A database is not the source project: DPH can inspect cached facts when the source is unavailable, but current-source review and source navigation require attached matching source. When the original entry is available, cached opening displays captured facts first and then verifies that source in the background.

Record the selected application profile

Choose Projects, then Selected project, to inspect captured project and group files and output declarations. Select a row to read settings, references, capture generation, source hash and source lines. Missing referenced projects remain visible. Project settings and their main-source declaration can describe the same application; this is an evidence list rather than an application count. Output kinds do not verify built binaries or installed services. Known databases returns to the remembered database list.

Open a saved project, choose Projects in the upper-right page selector, click Known databases and then Project profile. Enter the named contributor, purpose and scope, and attribution or evidence context. Fill the known company, department, business unit, criticality, ownership, support, environment, dependency, lifecycle, maturity, vendor, licensing and architecture fields. Unknown fields remain explicitly not supplied.

Click Apply to Knowledge draft. Inspect the generated body and click Save on Knowledge to persist it. Apply alone does not save. Reopen Project profile to edit its structured fields; the generated body is read-only. Saving never independently verifies the statements. Exact fields survive new revisions, named review, database reopen and canonical publication.

Define and review business terms

Open the intended project. In the upper-right workspace selector choose Business glossary, then click New term. Enter the business term, one alias or abbreviation per line, a plain-language definition, optional examples, relevance and evidence context, and a named contributor. Do not enter credentials. Context must explain why the term matters to this selected application and where its meaning came from.

Click Apply to Knowledge draft. This prepares an unsaved draft on Knowledge. Click Save and wait for the bottom activity line to finish. Return to Business glossary and select the saved term. Its state is needs_review until a person explicitly verifies it. Applying alone never saves or verifies a term.

To add exact source evidence, select the term and click Open in Knowledge. Choose Knowledge links in the upper-right selector, select the appropriate entity/evidence target and link role, then add the target. Return to Knowledge and click Save. Review the definition and its links against current source, enter a named reviewer in the author/reviewer field, then click Verify review. Source changes reopen review for affected linked terms.

To change a term, return to Business glossary, select it and click Edit term. Change its fields, Apply to Knowledge draft, then Save. A rename keeps the stable knowledge identity. Edits and reviews append history and require fresh verification. Open in Knowledge displays that history. Duplicate terms and aliases are rejected ignoring case and repeated spaces. Names belonging to retired records remain reserved; edit the existing term instead of creating a conflicting replacement.

The glossary filter finds term names, aliases, definitions and examples. Terms are alphabetized and review states remain visible. At a narrow window size, use the horizontal scrollbar under the glossary buttons to reach the filter. Drag the divider below the term list to change its height; scroll within the definition to read longer entries.

Publish the project with an appropriate scope/audience to a new folder outside selected source. Open index.html and find Business glossary; definitions also appear in both editable manuals, canonical.json and indexes/glossary.jsonl. The same captured term revision is used throughout. Confidential source exclusions remove dependent term content. Package validation checks the glossary index against the canonical model.

Repeatable check: create a synthetic term with an alias, Save, reopen the same project, find it by the alias, review it, edit its definition, Save again and confirm history and pending review. Try a duplicate alias and confirm rejection. Publish and compare the current definition and revision in HTML, both DOCX guides and the JSONL index. Use a test project, rather than inserting synthetic definitions into production knowledge.

Learn the selected project

Begin with Learn the project rather than trying to interpret the complete symbol index. Start here gives a six-step route: read actual startup statements, identify a familiar screen control, follow its event into code, inspect decisions and data, check gaps, then plan a controlled maintenance change. Each topic describes how to read the evidence and supplies project-specific starting points.

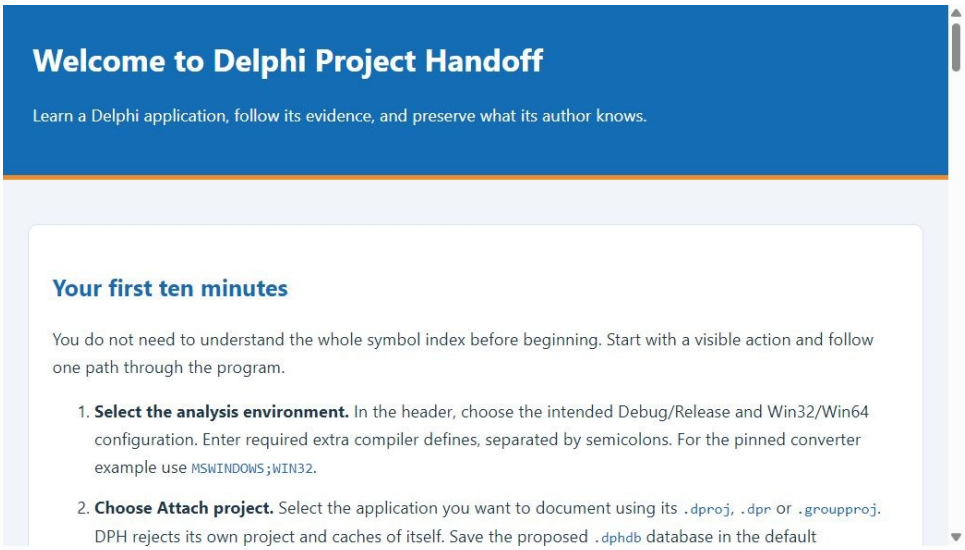
How the program starts quotes parsed program-body expressions with source locations and hashes. It does not substitute the first routine in a graph for the entry point. Screens and user actions links declared components. Follow an event explains binds, calls, callbacks and possible scheduling. Architecture, data and maintenance topics explain what their relationship kinds mean and where conclusions remain unsupported.

Click an Explorer object or follow a learning link to open Explain. It presents the declared contract, nearby source-author comments when captured, callers and consumers, outgoing interactions, decisions, attributed knowledge and a bounded execution trace. Source comments remain author documentation, not independently verified behavior. Expand a connection to see the precise evidence and derivation rule.

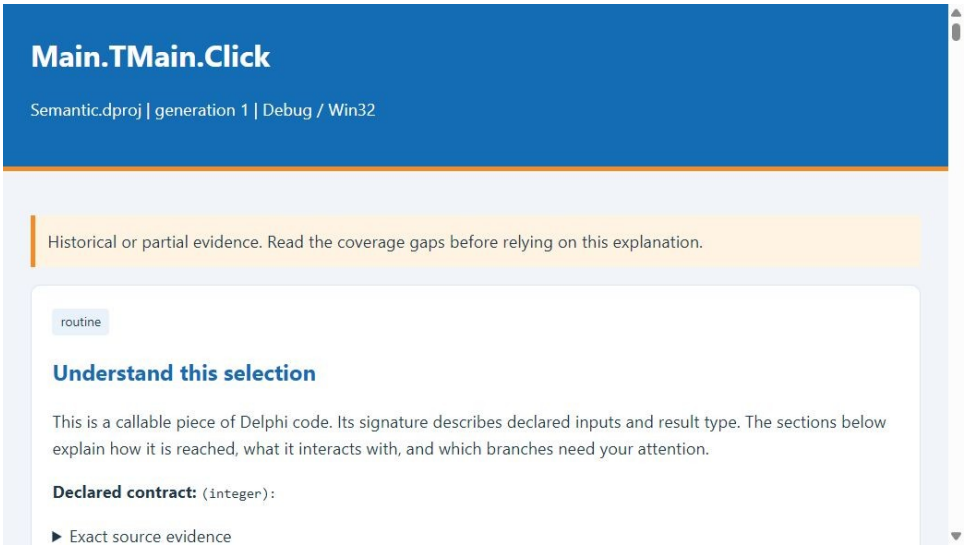
The learning and Explain pages run locally through WebView2. If the runtime is unavailable, a readable text fallback remains. Local learning projections avoid export-only redaction work; the publisher refuses to export them. A separately constructed publication model always applies redaction and exclusions.

The displayed learning pages

These images are direct WebView2 captures from the running DPH test application, not proposed interface mockups. They show the visible portion of scrollable pages.



Getting started: first-launch education before a project is loaded.



Explain: direct WebView2 capture for the synthetic semantics fixture. This is DPH rendering captured source evidence, not a screenshot of the analyzed application running.

Measure coverage before relying on an explanation

Choose Coverage in the page selector. Read each numerator, denominator and definition separately. Fingerprinted inventory measures acquired inputs; parsed files, complete IR and recognized tokens measure supported parsing. Confirmed and unresolved relationships measure the extracted graph. Serialized event coverage applies to captured On-prefixed properties. Evidence coverage applies to source-located relationships with derivation rules.

Select a diagnostic and inspect its source evidence. A missing unit, ambiguous overload or unsupported construct can change what a route means. None of these percentages measures business understanding. Verified authored revisions and pending review tasks are reported separately; a zero task count is not proof that all necessary documentation exists.

The selected application is not built or tested merely because DPH finished analysis. Read explicit unknown build and documentation approval states before sharing a handoff. Record the actual independent test procedure and result as attributed knowledge.

Explore and search

Click an object in the far-left Explorer list to select it. Double-click a [+] or [-] branch to expand or collapse its children. Selection details on the right shows its name, kind, signature, declared type, source location, generation and evidence hash. Choose Explain in the upper-right page selector for behavior and serialized properties when captured. Units, types, components and routines provide drill-down scopes. The breadcrumb reports the active scope and page. Parent moves upward; Back restores a prior selection.

Search matches object names, kinds and signatures across the project. A kind filter narrows the result. Pages contain at most 250 objects; Next page advances the result set. Clear the search or use Parent to broaden your route. Bookmark records the current canonical selection, and Bookmarks displays those selections. Same-project reassessment retains surviving bookmarks, history, expanded nodes and filters; missing identities are pruned. A different project resets that navigation state.

The References tab lists forward and reverse relationships with their certainty and derivation labels. Double-click a resolvable related object to inspect it. An unresolved target remains unresolved; the relationship is not proof that a named method actually ran.

Search across the captured project

Choose Search project in the upper-right page selector. Enter a meaningful term, select the desired kind filter and click Search. Results contain excerpts from captured object metadata, flow evidence and authored knowledge. Select a result to inspect it; use Next 50 results when more matches are available. This search is distinct from the narrower Explorer name and signature filter.

Search uses SQLite FTS5 when available and a word-index fallback otherwise. It operates on the captured generation. A word absent from the index is not evidence that its behavior is absent. Reassess changed source before relying on old results.

Read source and graph evidence

The Source tab reads the attached source file and compares its bytes with the captured SHA256 before applying the source range. If the live file changed or cannot be decoded, DPH displays a stale or unavailable

message instead of presenting the old range as a confirmed live location. External dependencies remain in the evidence model but are not followed by the local source viewer through an invented path.

The Graphs tab offers Execution, Procedure CFG, Caller / callee, Form navigation, Data flow, Change impact and Data lineage. Select a relevant routine or component before choosing a narrow graph. Ordinary graphs show at most 120 nodes and 960 edges. A narrower selection is preferable to interpreting a truncated large view as the complete program.

Confirmed static edges use blue. Uncertain edges use orange. Procedure CFG opens a labeled source-derived diagram when WebView2 is available, with a native graph fallback. Large routines collapse ordinary straight-line statements while retaining control nodes; arrows identify omitted statements. More than 70 retained control nodes produces an explicit limit instead of a misleading complete diagram. Other ordinary views retain their 120-node bound; lineage permits 200. Form navigation does not prove runtime window activation; Change impact is possible reachability. Click a native graph node to inspect its evidence.

Investigate source quality candidates

Choose Source quality in the page selector. Select Routine metrics, Duplicate bodies, Unreferenced candidates, Unreachable statements or Call cycles. Enter an optional name or source filter and click Filter. Select a row to read its meaning, source locations, hashes, generation and qualifications. Next 250 advances the bounded result page. Open explanation inspects a routine or the first group member.

Structural decision score counts captured decisions; it is not compiler-exact cyclomatic complexity or a maintainability index. Duplicate candidates compare nontrivial significant-token bodies with exact literals and normalized identifier case, only in completely parsed files. Equal token sequences do not prove equivalent behavior. Missing incoming references and disconnected captured statements do not prove that code is unused or safe to delete. Static call cycles do not establish feasible runtime recursion or termination.

An older detached capture may have no quality report. Attach matching source and reassess to obtain one. DPH retains the original evidence of detached caches rather than inventing new analysis from missing source.

Compare captured revisions

Choose Database tools and click Preview retention. Note an earlier generation ID with a retained workspace cache. Choose Revision comparison, enter that ID in Before: saved generation ID and optionally enter a name, relative path or change filter. Click Compare. After is the generation currently displayed by DPH.

Select a result. The labeled Before and After panes show declarations, source references, fingerprints and captured expressions or relationships. Drag the divider and scroll each pane to inspect the evidence. Next 200 advances the bounded result page. A missing cache produces an explicit error. Configuration and analyzer contracts must match.

File changed compares captured fingerprints and availability. Captured structure changed compares declarations, routine flow nodes and edges, and outgoing relationships including recorded event bindings. Source file changed only means the containing file changed while this symbol retained the same captured structure. Historical source bytes are not reconstructed. Partial parsing may miss changes, and removed or added identities require named continuity review before treating them as a move or rename.

Choose graph scope and grouping

Choose Graphs after selecting an object. Execution follows supported startup, event, call and asynchronous connections; Caller / callee isolates calls; Form navigation shows qualified form actions; Data flow shows producers, consumers and possible effects; Change impact follows possible dependencies. Procedure CFG follows captured statement edges within a routine.

Use Whole program, All forms or Project data when you need a broad starting point. Choose object, type or unit grouping to reduce a large relationship graph. Select a group to inspect its members and bounded evidence examples. A count describes the captured projection; it does not count observed executions. Procedure CFG retains individual control nodes.

Read arrow labels and certainty before interpreting a route. A unit import is a dependency, not a call. An asynchronous start, callback dispatch, wait or cancellation request describes a source site; it does not prove successful completion, thread timing or termination. Narrow a truncated graph rather than treating missing nodes as proof of absence.

Read a routine as a beginner

Select a routine in Explorer and choose Explain. Read its declared contract, source-author comments, incoming callers and outgoing interactions. Expand the exact-evidence disclosure for a connection you do not understand. A comment records what its source author wrote; it does not independently establish the running behavior.

Scroll to SOURCE WALKTHROUGH. Each source point includes the captured expression, a plain explanation and possible next paths with destination lines. For an if, inspect both true and false destinations. For while, inspect the condition before the body. For repeat, the body runs before its final test, and false returns toward the body. For for, inspect the iteration and exit paths without assuming actual bounds or collection contents.

Follow early return, break and continue through any intervening finally cleanup. Cleanup points may appear several times to preserve different pending transfers. An exception path represents a possibility; it does not establish that a particular run raised an exception. The walkthrough is arranged by source location and is not a chronological execution recording.

Complete CHECK YOUR UNDERSTANDING by choosing an actual captured condition and tracing both outcomes to a return, loop or exception boundary. Compare the related reads and writes. The lesson shows up to 60 source points and eight next paths per point, with explicit omitted counts. Use Source and Procedure CFG for the remainder.

Inspect screens

The Form designs tab reconstructs rectangles and text from serialized FMX or DFM data. It never creates untrusted component classes. The result is explicitly an approximate design-time view, not an actual runtime screenshot. Custom painting, runtime geometry, styles and unavailable inherited resources may differ from the running program.

Thumbnail displays a gallery of up to 60 forms. Fit scales the selected form to the viewport, Native uses its reconstructed size, Custom zoom ranges from 25 to 400 percent and updates the scrolling extent. Full screen

hides the surrounding workspace panes; Exit full screen or Escape restores them. Click a component rectangle to navigate to the corresponding canonical object when a source association is available.

Inspect keyboard shortcuts and focus order

Open the intended project and choose Keyboard and focus in the upper-right page selector. Select a component in the list. The detail box identifies its form, serialized parent, class, generation, source hash and line or byte location. It explains captured TabOrder, TabStop, CanFocus, visibility, enabled state, shortcut codes, action references and keyboard event bindings. Scroll inside the detail box to read diagnostics.

The list groups by file, form and parent, then sorts known sibling TabOrder values numerically. A question mark means unknown or not applicable, not zero. Missing properties can come from class defaults or inheritance. Duplicate sibling positions and possible duplicate shortcuts within a form are review candidates. Common Windows shortcut codes have readable labels such as Ctrl+S, while original numeric or hex codes remain visible. Repeated consumers of one action do not count again as duplicate shortcut declarations.

Use the filter to find a component, form, property or diagnostic. Next page advances through at most 250 components per page; First page resets. Scroll the toolbar horizontally in a narrow window to reach the source commands. Component source navigates to verified attached form text. Action source opens a uniquely resolved local action declaration. Detached caches, changed source, historical generations, binary text offsets and paths outside the attached project are refused with an explanation.

DPH does not run the target program. Runtime code, nested containers, inherited values and framework behavior can change keyboard navigation. External or ambiguous actions remain unresolved. Secondary shortcut collections are retained as raw evidence and explicitly excluded from decoded conflict checks. Run the target yourself to check Tab, Shift+Tab and its declared shortcuts. Complete directions are in docs/guides/Keyboard_and_Focus_Run_and_Test.txt. Developers can run tools\build-keyboard.cmd from an elevated Command Prompt.

Inspect menus context menus and toolbars

Open the intended project and choose Menus and commands in the upper-right page selector. Select a command to read its serialized menu path, caption or Text, action reference, shortcut codes, check flags, source location and diagnostics. The page inventories main and context menus, menu items and separators, toolbar commands and children, action containers, action consumers and menu attachments.

Choose a command kind or enter a name, caption, property, handler or diagnostic in the filter. Next page advances through 250-row pages; First page resets. Scroll the command bar horizontally to reach Command source, Action source, the handler selector and Explain handler. The detail box below the list scrolls vertically. Drag the divider below the list to resize it.

Command source opens unchanged attached form text. Action source opens a uniquely resolved local action declaration. Select the specific component OnClick, action OnExecute or OnUpdate, or context OnPopup in the handler selector, then click Explain handler to follow its confirmed captured routine. An unresolved handler disables that button. The separate bindings do not establish dispatch precedence. Source navigation refuses changed, detached, historical, binary or out-of-project text locations.

Menu paths follow serialized parents, not observed screen positions. An ampersand in a caption declares an accelerator marker; a separator is not executable. Missing properties remain unknown. OnUpdate and OnPopup can change state and contents at runtime. Readable Windows shortcut names and possible

conflicts are on Keyboard and focus. Use docs/guides/Menus_and_Commands_Run_and_Test.txt for explicit run and test steps. DPH does not execute the target application.

Inspect dynamic and runtime created controls

Open the intended project and choose Dynamic controls in the upper-right page selector. Attach the source and click Full reassessment for an older capture; DPH-WP3-15 adds owner, parent and event relationships that old captures do not contain. Select a row to read its constructor class, generation, source range and fingerprint. Each row is a source creation site; a loop does not establish its runtime instance count.

The detail box separates the lexical result variable, constructor owner, visual Parent assignment, event assignment and other property expressions. Owner describes lifetime ownership; it does not set Parent. Self refers to the declaring type, not an observed instance. Inferred links identify source declarations. nil preserves an explicit empty-reference expression. A reused or overwritten variable or assignment before construction remains ambiguous. Missing references remain unknown.

Filter by routine, class, parent, owner or event. Next page advances through 250 sites and First page resets. Scroll the command bar horizontally to reach Explain creation site, the reference dropdown and Explain reference. Choose a relationship before explaining its target. Unresolved targets and nil expressions disable reference navigation. Explain opens captured evidence; use Source there for verified attached code. Scroll the detail box vertically or drag its divider to resize it.

DPH does not execute the target or reconstruct observed controls from these sites. Runtime instances, final layout, dispatch and lifetime remain unobserved. Factories, aliases, casts, with scopes and cross-routine mutations can remain unsupported. See docs/guides/Dynamic_Controls_Run_and_Test.txt for explicit directions and manual exercises. Developers can run tools\build-dynamic-controls.cmd elevated for the focused acceptance suite.

Inspect responsive layouts anchors and alignments

Open the intended project and choose Layout rules in the upper-right page selector. Select a component to read its captured Align, Anchors, geometry, margins, padding, constraints, scaling and resize events with source locations. Parent means the serialized containing object. Missing and inherited rules remain unknown. These explanations help you investigate resizing without executing the target application.

Enter a parent width change and height change, then click Apply example. Positive values illustrate growth; negative values illustrate shrinking. Each field accepts a whole number from -10000 to 10000. Invalid input preserves the previous valid example. At narrow widths, scroll the command bar horizontally to reach these fields and Apply example. Applying refreshes the filter and returns to the first page.

Numeric anchor examples require an ordinary serialized child, a captured parent, explicit Align=None and recognized explicit anchors. Opposite edges illustrate a size change; trailing edges illustrate movement; leading edges illustrate a fixed offset. No anchor on an axis leaves its framework anchor origin unknown. Active alignment, inherited objects and missing or unknown rules receive explanations without unsupported numeric predictions.

Filter by component, property or explanation. Next page advances through 250 rows; First page resets. Drag the divider below the list to resize it and scroll the detail box for all explanations. Component source opens attached unchanged text at its captured location. Detached, changed and binary source locations are refused explicitly.

The example reports basic changes, not absolute final geometry. Constraints, autosizing, sibling alignment, inherited defaults, spacing, fonts, DPI and custom code can alter the actual result. Run the target separately and resize it at narrow and wide widths and different display scales. Compare with its Object Inspector settings. The Screens reconstruction remains approximate. Full directions are in docs/guides/Layout_Rules_Run_and_Test.txt; developers run tools\build-layout-rules.cmd elevated.

Inspect component dependencies and support records

Open the intended project and choose Component dependencies in the upper-right page selector. Select a component to read its class, form source, generation and fingerprint. DPH lists all captured classes because a name alone cannot establish third-party origin. Unresolved means no class declaration was captured; it can involve framework, vendor, local, inherited or excluded source.

A unique class declaration is an inferred lexical candidate. Multiple declarations remain ambiguous. Explain class opens the unique candidate in Explain; it is disabled when no unique candidate exists. Component source opens attached unchanged form text. Detached, changed and binary source locations are refused explicitly. Captured package declarations and unavailable inputs are shown as project-wide context; they do not establish package membership or installed binaries.

Click Edit dependency for the selected class. Enter a named contributor, declaring unit, package or product, vendor or maintainer, version and configuration, support or replacement considerations, and evidence references with remaining unknowns. Class identity is fixed. Evidence and contributor are required; blank optional fields remain unknown. Scroll the editor vertically to reach every field. Click Apply to Knowledge draft, then click Save on Knowledge and wait for completion. Applying alone does not save.

Open in Knowledge shows the saved revision and immutable history. Use Knowledge links to add exact evidence and save, then enter a named reviewer and click Verify review only after checking the claims. Editing dependency fields creates a new revision and reopens review. Duplicate class records are rejected ignoring case; retired identities remain reserved. Save or discard an existing draft before navigating or editing another record.

Filter by class, resolution or authored vendor/support wording. Next page advances through 250 rows; First page resets. Scroll the command bar horizontally at narrow widths and drag the list divider or scroll inside the details. Authored metadata never changes source-resolution status. Attach approved missing source roots and reassess to obtain declarations. No vendor class, package or DLL is loaded. Full directions: docs/guides/Component_Dependencies_Run_and_Test.txt.

Inspect inherited forms and embedded frames

Choose Form inheritance in the upper-right page selector after opening the intended project. Select a component to read its declaration origins and base-to-derived property/event history. Captured base-only components remain visible. Inline frames expand independently for each instance, including inherited frame definitions. Root names can differ between ancestor and descendant.

Each property shows its latest serialized value and every captured earlier value with source, line, SHA256 and generation context. An explicit nil event clears an inherited binding; this does not establish actual event dispatch. Read Resolution gaps for missing or ambiguous bases, orphan or incompatible child overlays, conditional ancestors, partial inputs and cycles. Framework defaults, collection-item merging and runtime changes remain unknown.

Scroll the command bar horizontally to reach the origin selector. Select a component declaration or a specific property/event value, then click Open origin source. A base value opens base source; an override opens its overriding source. Navigation requires attached unchanged text from the captured generation. Detached, changed, binary and outside-root origins are refused explicitly. Binary byte provenance remains visible.

Filter by component path, property, value or gap. Next page advances through 250 rows; First page resets. Drag the list divider or scroll the explanation to read more. Full executable paths, exact steps, fixture examples and elevated developer checks are in docs/guides/Form_Inheritance_Run_and_Test.txt. This inspector combines captured declarations; other views retain their own captured-evidence qualifications. No target form/frame/class is instantiated.

Import actual screenshot evidence

Open Screenshots, enter the named contributor and a description including capture date and environment, then choose Import an existing PNG screenshot. Select an image you already captured and are authorized to preserve. DPH does not launch the selected application. PNG files are bounded to 20 MiB, 8192 pixels per dimension and 32 million pixels.

The managed image and its metadata are checksummed beside the knowledge database. Select a row to view the actual image and attribution. Runtime origin, capture time and environment are contributor claims and remain user supplied unverified. The associated generation is displayed, including when it is historical. Copy the adjacent views folder when moving the database.

Publishing includes only imported images matching the exported generation and included object scope. Excluding a linked source excludes its screenshot. The exported caption identifies the contributor and unverified state; private storage paths are omitted. Review screenshots for confidential material before sharing.

Inspect database connections and datasets

Open the intended project or saved database and choose Database connections in the upper-right page selector. The inventory lists captured connections, datasets, data sources, transactions, sessions and named connection-definition references. Known component class names and database property signals identify candidates. Pascal-only declarations are also listed when an existing semantic component-field binding does not already represent them. This is captured evidence; DPH never opens the selected application database or loads its drivers.

Select a row to read its role, class, generation, source line and SHA256, masked configuration, captured references and Used by dependents. Scroll the evidence memo vertically. Drag the divider below the list to give the list or memo more space. The kind selector and text filter narrow the inventory; text also searches references and reverse dependencies. First page and Next page show 250 rows at a time. Scroll the toolbar horizontally in a narrow window.

Use the reference selector on the right of the toolbar to choose a Target or Used by entry, then click Follow reference. DPH clears the filters and opens the exact referenced row and page. Unique local, nested and explicitly qualified cross-module references are lexical candidates, not proof of runtime binding. Duplicate names, wrong component roles, missing targets and unsupported expressions remain unresolved. Unqualified references are not guessed across unrelated forms. Explicit nil stays explicit.

ConnectionDefName creates a named definition-reference row with reverse references to its connections. Definition contents, engine/version and environment overrides remain unavailable; no INI, registry or external configuration file is silently read. BDE DatabaseName and session aliases can link to captured declarations. Pascal-only declarations and inherited components do not establish runtime construction, assignments or defaults. An empty result does not prove that the program never uses a database.

Configuration is deliberately masked, including endpoints, database paths, users, passwords, connection strings and unknown parameter values. Only validated driver/protocol/dialect/charset/SSL identifiers, ports and booleans are displayed. Connected=True is a serialized declaration, not evidence of successful connection. Source and cache files still contain original evidence and may contain secrets; the inventory is not a scrubbed database backup. Ask the maintainer for approved environment and definition facts without putting credentials into authored explanations. Full paths, practice steps and developer checks are in docs/guides/Database_Connections_Run_and_Test.txt.

Inspect database engines and versions

Open the intended project or cache and choose Database connections from the upper-right selector. Each row now summarizes Engine and Declared version. Select it to read DATABASE ENGINE AND VERSION EVIDENCE at the top of the labeled memo. Drag the divider below the list to resize the evidence area and scroll vertically for the full explanation. Use the text filter to find an engine family such as Firebird, PostgreSQL or SQLite, or a component name. General explanatory text is excluded from matching so it does not make every row match an engine name.

A recognized captured FireDAC DriverID identifies a declared engine family, not a successful runtime connection. Supported IDs include ADS, ASA, DB2, FB, IB, IBLite, Infx, MSAcc, MSSQL, MySQL, Ora, PG, SQLite and Mongo. MySQL-compatible deliberately does not distinguish MySQL from compatible servers. Generic component classes, ODBC, virtual driver names, filenames, ports, SQL dialect and the version of DPH own SQLite store cannot identify the selected project database. Unrecognized drivers remain unknown; contradictory driver declarations remain conflicting.

ServerVersion and DatabaseVersion show only bounded numeric declarations actually captured in properties or parameter entries. The label declared means that DPH has not measured the server. ClientVersion and DriverVersion remain separately named evidence and never become server versions. Conflicting numeric declarations are reported as conflicting. Missing, invalid or sensitive values remain unknown or masked. A deployment can override every design declaration; compatibility and current availability remain unknown.

A dataset, transaction or data source can inherit engine evidence through unique resolved captured connection or dataset links, up to three hops. The memo retains the reference path and original connection source, line, generation and SHA256. Conflicting targets and unresolved paths do not invent an engine. Named external connection definitions stay unknown until their actual configuration is captured; DPH does not inspect registry, INI files or live servers. Follow reference navigates exact captured targets as before. Full executable paths, practice examples and developer checks are in docs/guides/Database_Engines_Run_and_Test.txt.

Inspect tables views and schemas

Open the intended project or cache and choose Database schema in the upper-right page selector. A table stores rows, a view names a query result, and a schema qualifies object names. DPH shows what captured source declares or references; it does not connect to a database. Select a row to read its name, evidence

state, schema qualifier, captured definition, source line, SHA256 and generation in the labeled evidence memo. Drag the divider below the list and scroll the memo for the full explanation.

Declared header means a supported CREATE TABLE, CREATE VIEW or CREATE SCHEMA header was captured in a referenced SQL resource with balanced parentheses. Its displayed definition omits comments and string literals. This establishes the captured declaration text, not valid body grammar, column or constraint semantics, deployed existence, privileges or database compatibility. Qualifier reference means a qualified object name mentions a schema; it does not establish CREATE SCHEMA. Inferred reference means the existing supported SQL analysis names an object whose table or view identity and definition are unverified. Unsupported statements remain visible as gaps.

Combine the object-kind selector, text filter and evidence-state selector to find a table, view, schema or unsupported statement. Text matches object names, qualifiers and source files. First page and Next page show up to 250 matches. In a narrow window, scroll the command strip sideways to reach the evidence-state selector and Open source. Open source requires attached, unchanged source from the same generation. Detached, changed, historical or outside-project source is refused with an explanation in the memo.

Multiple declarations with the same exact kind and spelling retain every definition and show a count. DPH does not choose a migration winner, merge dialect-specific identifier case, or infer deployment order. Ask the maintainer which approved scripts and migration sequence define the intended system. Empty results can mean missing resources or filters; they do not prove there is no schema. Referenced literal SQL files are discovered through existing project inventory rules. Original source and caches remain unchanged and can contain sensitive data. Full start, practice and developer directions are in docs/guides/Database_Schema_Run_and_Test.txt.

Inspect column types defaults and constraints

Choose Database schema in the upper-right page selector after opening the intended project or cache. Select column in the kind selector and declared column in the evidence-state selector. Select a row to read Declared type, Nullability declaration and Default declaration at the top of the labeled memo. Columns are the named fields declared within a table. A data type describes the declared representation of a field; a default is a captured expression used when an applicable database operation omits a value; a constraint states a rule. DPH captures those declarations without executing or validating them against a server.

Explicit NULL and explicit NOT NULL identify clauses actually present in that column entry. Not declared means absent from that entry, not an inferred effective default or permission to store nulls. Primary keys, domains, inherited schema and later migrations can affect effective rules; their implications are not synthesized into explicit declarations. DEFAULT NULL is a default expression and does not become an explicit nullability clause. Numeric, symbolic, simple function and parenthesized defaults retain captured text. String literal contents are replaced by a visible literal omitted marker. Nothing is evaluated.

Inline constraints show captured NOT NULL, NULL, PRIMARY KEY, UNIQUE, REFERENCES and CHECK text, including supported named clauses. Choose constraint and captured constraint to read table-level entries separately. A CHECK expression can contain commas without splitting the table incorrectly. Constraint text alone does not establish key membership. The key view described in Inspect primary foreign and unique keys now captures supported membership and reference evidence. Valid expression semantics and deployed enforcement require separate investigation. Every column retains the original source line, SHA256, generation and exact parent table declaration identity, so duplicate table definitions cannot silently merge their columns.

Filter by table or column name, source file, declared type such as DECIMAL, explicit nullability or inline constraint text. Combine this with kind and state filters. First page and Next page show 250 matches; scroll commands sideways in a narrow window. Unsupported column clauses, missing types/default expressions and conflicting explicit nullability become gaps with the full sanitized entry and an explanation. Later supported entries remain inspectable. Open source uses the existing attached-source hash and generation guards. Read docs/guides/Database_Columns_Run_and_Test.txt for complete directions and controlled practice.

Inspect primary foreign and unique keys

Open your captured project or cache, then choose Database schema in the upper-right page selector. Select key in the kind selector and declared key in the state selector. A primary key declares how to identify a row, a unique key declares a distinct combination of column values, and a foreign key declares a reference to another table. These are captured declarations; DPH does not connect to your server or establish that the rules are deployed.

Select a key row. The memo begins with Key type, Ordered local columns, Referenced table, Ordered referenced columns and Relationships. Composite columns retain their written order. Named constraints retain their name; anonymous constraints use a source-local label, not a fabricated server name. Inline PRIMARY KEY, UNIQUE and REFERENCES declarations become single-column key records. Supported table constraints capture composite lists and ON DELETE or ON UPDATE actions. A primary key does not rewrite the separately captured explicit nullability of a column.

Scroll the command strip sideways to reach the relationship selector and Follow relationship. Choose owning table, local column, referenced table or referenced column, then click Follow relationship. DPH clears the filters and opens that exact captured row, including rows on later pages. Only uniquely matched evidence appears as a navigation choice. Missing or ambiguous declarations remain explained in the memo. An unqualified reference is not matched to a guessed schema; quoted names and case retain their exact captured spelling.

Relationship gaps include a missing column or table, duplicate declarations, omitted referenced columns, differing local and referenced column counts, duplicate constraint names and multiple primary keys in one declaration. DPH does not guess omitted primary-key targets or select an effective migration. Captured links complete means every requested evidence link was identified; it does not validate target uniqueness, data compatibility or database enforcement. Unsupported key expressions and tails remain explicit gaps. Inline reference actions beyond the supported column reader remain column gaps.

Filter by constraint name, table, source, key type such as foreign, referenced table or relationship status. First page and Next page show 250 matches. Open source requires the attached unchanged source and matching generation. Read docs/guides/Database_Keys_Run_and_Test.txt for explicit launch, practice and validation steps. The controlled fixture includes Child_FK referencing Parent, along with deliberately missing and ambiguous examples.

Inspect indexes and performance evidence

Open the intended project or cache and choose Database schema in the upper-right page selector. Choose index in the kind selector and declared index in the state selector. Select a row. An index is a declared access structure that can affect reads, writes and storage. DPH shows what captured source declares; it does not connect to a database or measure whether the index helps.

The memo starts with Indexed table, Unique declared, Method, Storage kind and Ordered index members. Member order, explicit ASC or DESC and NULLS FIRST or LAST are retained. Not declared means absent from the captured declaration; it does not imply a default for an unknown database engine. Function or parenthesized expressions retain sanitized text. Their column dependencies remain unverified rather than being guessed from words in the expression.

Included columns are separate from key members. A predicate records the captured text of a partial or filtered index. Unique null treatment, explicit clustering, CONCURRENTLY, IF NOT EXISTS, ONLY, WITH storage options and TABLESPACE declarations remain separate facts. Text inside predicates and storage options is not evaluated or validated as database syntax. Literal values are omitted from the projection. Duplicate definitions remain separate, with the effective migration unresolved.

Scroll the command strip sideways to the relationship selector. Choose indexed table, indexed column, included column or SQL table reference and click Follow relationship. DPH opens the exact captured target and its required 250-row page. Missing or ambiguous tables and columns have no invented link. A SQL table reference means supported captured SQL names the same exact table; it does not establish that a query used this index or that its predicates or selected columns are covered. Zero references means no supported captured match, not that the index is unused. Navigation shows up to 250 references per index and 10000 across the projection, with total and shown counts.

Filter by index name, target table, method, predicate text, clustering, storage options or relationship status. First page and Next page show 250 matches. Unsupported member options, missing names or lists, malformed structures and unsupported tails remain explicit gaps. Open source uses attached-file generation and hash guards. Follow docs/guides/Database_Indexes_Run_and_Test.txt for launch, practice and developer validation steps. In the controlled fixture, IX_Composite links Data.Target, ordered a and b members, included note and a separate SQL resource.

For a performance investigation, ask the maintainer for an approved execution plan and measured workload observations. Record supplied runtime evidence through the existing Runtime observations workflow with contributor and run context. Source declarations and matching SQL references alone do not establish optimizer selection, physical size, timings, selectivity or an index recommendation.

Inspect stored procedures functions and triggers

Open the intended project or cache and choose Database schema in the upper-right page selector. Choose procedure, function or trigger in the kind selector, then declared stored object in the state selector. Select a row. A procedure groups database work, a function declares a returned value and a trigger associates work with a database event. DPH inspects already captured SQL resources; it does not execute the selected application or SQL.

The details memo begins with the stored-object name, captured CREATE operation, language, return type and ordered parameters. Each parameter shows position, exact name, IN/OUT/INOUT or supported output mode, declared type, default text and source line. Literal default values are replaced with a literal-omitted marker. Not declared means absent from the captured statement, not an engine default or proof that a value cannot be returned.

For a trigger, read Trigger table, Timing, Events, Scope and WHEN before the body. BEFORE, AFTER and INSTEAD OF describe captured timing; they are not observations of an execution. A supported trigger can

retain an inline block or an explicit zero-argument function call. WHEN remains unevaluated text. Unsupported event or option grammar stays in an explained gap.

Scroll the command strip sideways to the relationship selector, choose a table reference, procedure call, trigger table or function call, then click Follow relationship. DPH clears the filters and selects the exact captured target on its required 250-row page. Missing or ambiguous definitions have no guessed link. Multiple definitions and overloads remain independent; a name match cannot establish runtime argument binding or which migration is effective.

The captured body appears after the structural facts and relationship notes. Comments and string literals are omitted, including literals inside supported single-quoted or dollar-quoted SQL bodies. The original attached source or cache remains unchanged. Open source requires the current attached capture generation and unchanged source hash; detached or changed source produces an explanation instead of opening unverified live content.

Dependency coverage is deliberately qualified. Supported literal table references and explicit calls provide evidence links. Dynamic execution, expression calls, aliases, CTE binding and column binding remain unverified; table references in a body containing WITH stay unresolved. Empty dependencies do not establish independence. Display is bounded to 1000 dependencies per object and 10000 across the projection; a reached limit is explained. These are source relationships, not observed execution paths.

Search accepts object name, source, language, return type, trigger event/timing, parameter text or sanitized body text. First page resets pagination; Next page shows the next 250 matches. For a practice session and exact launch/build directions, use docs/guides/Database_Stored_Objects_Run_and_Test.txt. In the controlled fixture Data.Save has ordered parameters and links to Data.Target, Data.Log and Page270. Data.Audit links its event table and Data.Hook function. MissingRef, DynamicRef and AmbiguousHook demonstrate evidence gaps.

Inspect entity relationship diagrams

An entity relationship (ER) diagram shows tables as boxes and declared foreign keys as arrows. Choose Database ER diagram in the upper-right page selector after opening the intended project or cache. DPH uses captured SQL table and key metadata. It never connects to a database server or executes target SQL. Empty results mean no matching supported captured declarations, not proof that the application has no database.

Read PARTIAL schema above the drawing. Its counts show matching tables, related foreign keys, omitted relationships, source gaps and unresolved foreign keys. Blue arrows connect captured exact table targets; orange dashed arrows mark unresolved relationships when both endpoints are available. Missing or off-page targets remain in the FK evidence selector without an invented box or line. Cardinality, deployed enforcement, referenced uniqueness, type compatibility and effective migration order are unverified.

Each table box shows ordered column names and declared types. PK means primary key, FK means foreign key and UQ means unique key. These badges describe captured column membership. A repeated membership appears once per badge. Orange table borders mark gaps, duplicates or a truncated column summary. The footer reports shown and total columns. Click a table to read its full captured columns, keys and gaps in the labeled details memo below the drawing. Open evidence selects that exact table in Database schema, including its required page.

To inspect an arrow, scroll the command strip sideways to the FK evidence selector and choose its numbered entry. The memo shows the original key, ordered local and referenced columns, source hash, capture

generation and source range. Open evidence selects the exact key in Database schema. Arrow numbers correspond to selector entries. The selector retains relationships touching a visible table even when the other endpoint is outside the focus, page or filter.

For a smaller diagram, select a table, column or key in Database schema, then click View ER diagram in that page's command strip. Alternatively click Focus schema selection in the ER command strip. Focus includes that exact table and its directly linked incoming and outgoing neighbors. The text filter searches table name, schema and source within that focus. All tables clears both focus and text. First page resets paging; Next page advances by the selected 15, 30 or 60 table page size. Zoom offers 75 through 200 percent; use the drawing scrollbars to reach enlarged cards and lower rows.

The default IDE-editable cards summarize seven columns; all captured columns remain available in table evidence. A page holds at most 60 tables and 120 related foreign-key entries. Omitted FK counts are explicit. Duplicate table declarations stay separate; inferred SQL references, views and unsupported statements do not become table entities. The diagram is regenerated when a capture is replaced. In a small window, scroll commands sideways, scroll the drawing independently and drag the details divider if needed. Maximize the window for the largest useful diagram.

For exact launch, practice and build steps, read docs/guides/Database_ER_Diagrams_Run_and_Test.txt. The controlled fixture contains ER_Customers, ER_Orders and ER_Lines with two arrows. Focus ER_Orders to see that three-table chain. ER_ManyLinks deliberately exercises the relationship display limit. Inspect SelfRef and MissingRef for self links and unresolved evidence. These diagrams are rendered by the running native application, not mockup screens.

Define table and column business meanings

A data dictionary explains what a table or column means to the people using the application. DPH captures technical SQL structure, but a knowledgeable contributor must supply business meanings. Open Database schema in the upper-right page selector. Select a declared table or column. The labeled details area starts with its saved business dictionary, or explains that none has been recorded for these captured source bytes.

Scroll the command strip horizontally to the right and click Edit dictionary. Enter the named contributor, Business definition, Business owner or steward, Sensitivity and handling, and Evidence and remaining unknowns. These fields are required; explicitly write Unknown and the person or evidence needed when an answer is unavailable. Units values and business usage is optional. Do not paste actual customer records, credentials or confidential example values. A sensitivity entry is documentation, not automatic access control.

The editor begins with business fields. Scroll downward to inspect the read-only schema identity, source path, SHA256, original capture generation, object kind/name, owning table and statement line. These identify the exact captured declaration; duplicate table names are not merged. DPH does not infer a business definition, responsible owner or sensitivity from an SQL name or declared type.

Click Apply to Knowledge draft. The editor closes and Knowledge displays the protected structured draft. Click Save to create an immutable revision. Apply alone does not persist anything. Use Discard edit if you do not want the draft. To change a saved definition, return to Database schema, select the same table or column and click Edit dictionary again. Editing requires a fresh human review.

Click Open dictionary history beside Edit dictionary to view contributor, editor, review state and immutable revisions in Knowledge. Enter the named reviewer and choose Verify review, Reject or Retire as appropriate.

Saving a definition does not verify its truth. Additional canonical source, relationship, screenshot or section evidence can be added on Knowledge links. Open source on Database schema follows the selected captured source under the existing hash checks.

The Database schema text filter also searches saved current definitions, owners, sensitivity and usage. Clear the filter to restore the inventory. Tables or columns not captured as supported declarations cannot be given invented schema identities. A changed SQL resource does not automatically inherit its old dictionary. When the same source/record/name has historical text, details explains the mismatch and Open dictionary history preserves access. Other unmatched records remain in Knowledge. Recheck business facts and author a definition for the newly captured declaration.

Project handoffs include matching attributed dictionary records through the existing Knowledge sections and structured canonical records. Source exclusions and selected source scope apply; definitions whose captured source hashes differ are omitted. Review business descriptions before publishing. Native controls remain editable in the Delphi IDE. No SQL, target application or database server is run by this workflow.

Follow dataset and screen field bindings

Open the project or cached database, then choose Database connections in the upper-right page selector. Choose bound control in the kind filter to find data-aware controls, persistent field for captured field components, or data source for the intermediary between a control and dataset. Clear the text filter before changing the kind if no rows appear. Field names and component paths are searchable.

Select a bound control. The labeled details memo begins with DATASET AND SCREEN BINDINGS and lists its captured DataSource and DataField references. Select Target via DataSource in the reference selector and click Follow reference to inspect the data source, then follow DataSet to its dataset and Connection to the captured connection. To go straight to a captured persistent field, select Target via DataField and click Follow reference. A field name can refer to a calculated or lookup field; it does not prove a live database column.

Select a persistent field to see its captured FieldName, FieldKind, Required, ReadOnly, Size and Precision when serialized. Its DataSet reference identifies the serialized owning dataset, and Used by references return to bound controls. Choose a reference in the selector before clicking Follow reference; use the horizontal toolbar scrollbar when the command is outside the visible area. The details memo scrolls independently.

For FMX TLinkControlToField, DPH recognizes the serialized DataSource, FieldName and Control properties. Follow DataSource to TBindSourceDB and DataSet, FieldName to a unique persistent field, or Control to the captured visual target. This explains stored component references without creating any target component or executing LiveBindings. General expression-based bindings, dynamically created fields, runtime assignments, uncaptured inherited values and missing external targets remain unknown.

Duplicate field declarations or ambiguous data-source targets are reported without choosing one. A missing persistent field remains unresolved even if a runtime dataset might create it. Native controls are editable in the IDE. Run and test directions are in docs/guides/Dataset_Bindings_Run_and_Test.txt. The screen-data inventory is evidence for the selected captured revision, not a live database connection or an assertion about production data.

Inspect database read and write operations

Choose Database connections in the upper-right page selector, then choose database operation in its kind filter. Clear the text filter to see all captured requests; enter Post, Delete, read, insert or a calling routine

name to narrow the list. Select a row. The evidence memo begins with its operation kind, requested effect and evidence basis. Operation requests are source candidates; DPH does not run the selected application or connect to its database.

Open, Refresh, navigation and search methods are read candidates. Append and Insert prepare a new record buffer; Edit prepares an update. AppendRecord and InsertRecord request insertion. Post submits a pending insert or edit, depending on the dataset state; cached updates can defer persistence. Delete requests deletion of the current record. ApplyUpdates can submit mixed pending writes. Cancel and CancelUpdates concern pending edits or cached changes. None of these calls proves transaction commit or rollback.

Literal SELECT, INSERT, UPDATE and DELETE text is classified separately from calls. Configuring SQL does not execute it. Joined statements retain their lexical family even when table and column binding is unsupported. Dynamic SQL, mixed batches and unsupported prefixes remain unknown. SQL values are omitted from this inventory. Inspect approved source locally to understand parameters and context.

Select Target via DataSet in the reference selector and click Follow reference to inspect the exact captured receiver. A dataset lists reverse operation references in the same selector. Click Source to open the original location only after attaching unchanged source; detached, changed and historical evidence refuses navigation. Use the horizontal toolbar scrollbar to reach Follow reference and Source when they are outside the visible area. Scroll within the evidence memo for engine evidence and unknowns. Full reassessment upgrades an older capture to the new analyzer. Explicit run and test directions are in docs/guides/Database_Operations_Run_and_Test.txt.

Inspect transaction and rollback paths

Choose Database connections in the upper-right page selector. In its kind filter choose transaction operation, then clear the text filter. Select StartTransaction or BeginTrans to inspect a begin request. Select Commit, Rollback or a retaining variant to inspect that boundary. Use the calling routine name to narrow the list. Searches include linked references, so a search for a begin call may also show its connected commit and rollback rows.

The evidence memo begins with TRANSACTION PATHS (SOURCE POSSIBILITIES). Each route leads to the next explicit transaction request on the same declared receiver, or to a normal/exception routine exit without another boundary. Possible normal and exception paths remain distinct. Conditional/loop routes and duplicated finally cleanup copies are labeled. A route is a conservative source possibility, not proof that the transaction started or that a commit or rollback succeeded.

In the reference selector choose a Target via possible normal path or possible exception path, then click Follow reference to inspect the next boundary. Target via Transaction opens the captured receiver; Used by returns to an earlier boundary. Follow reference clears filters to reveal its exact target, so reselect transaction operation when returning to the inventory. Scroll the command bar sideways to reach Follow reference and Source. Source requires unchanged attached source; a detached cache refuses navigation.

CommitRetaining and RollbackRetaining preserve or restart transaction context according to the provider. RollbackSavepoint concerns a savepoint rather than necessarily the full transaction. Post and Cancel are dataset operations and do not establish transaction completion. A visible exit without another boundary requires source and caller review; implicit transactions, aliases, external effects and provider behavior remain unknown. Missing, ambiguous or truncated flow must never be read as safe completion. Explicit launch and test steps are in docs/guides/Database_Transactions_Run_and_Test.txt.

Inspect field validation and transformations

Open the intended project or cache and choose Database connections in the upper-right page selector. Choose data rule in its kind filter. Clear the text filter, then search for a field, handler, predicate, constraint or transformation expression. Select a row. The labeled evidence memo begins DATA VALIDATION AND TRANSFORMATION EVIDENCE and shows Input expression, Output expression, Constraint, Captured expression and conditional compilation context. Drag the divider below the list or scroll the memo for the full explanation.

Declared field constraints include captured Required, ReadOnly, Size, Precision, MinValue, MaxValue, CustomConstraint, DefaultExpression, error-message and edit-mask declarations. Numeric and constraint structure remains visible; literal contents are omitted. A declaration does not establish an inherited default or a successfully enforced rule. Default expressions are not evaluated. An omitted message or mask can be inspected through approved hash-checked Source.

Validation and conversion hooks include OnValidate, OnSetText, OnGetText, BeforePost and other recognized dataset or input events. Captured handler conditions, raises, early returns and assignment expressions provide technical rule candidates. Assignment rows identify the written expression and its input expression, with links to available lexical declarations. Handler context identifies its associated field, dataset or bound control; it does not prove that every statement acts on that field or that an event actually ran.

Scroll the command bar sideways to reach the reference selector, Follow reference and Source. Follow rule context to the field or dataset, handler to its captured routine, or lexical input/output to a declaration. Follow clears filters to show the exact target; reselect data rule afterward. Source requires unchanged attached source in the displayed generation. Missing or ambiguous handlers and unavailable syntax remain explicit. The shared 10000-row inventory limit reports omitted rules. Full launch, practice and test directions are in docs/guides/Data_Rules_Run_and_Test.txt. DPH does not run target code, SQL or validation.

Trace inputs and destination candidates

Start DPH from bin/Win32/Debug/DelphiProjectHandoff.exe. Attach the application you intend to document or open its captured database. Choose Database connections in the upper-right page selector. Choose bound control, persistent field, data rule or database operation in the kind selector and select a relevant row. Drag the horizontal toolbar scrollbar fully right to reveal Both directions, Trace lineage and Lineage diagram.

Click Trace lineage. The labeled memo shows the starting point, direction, capture generation and truncation state, followed by scoped route nodes, captured edges and unresolved breaks. Each node identifies its captured source location and SHA256. Select a Lineage hop in the reference selector, then click Follow reference. A component or rule opens its inventory evidence; a semantic declaration or file output opens Explain. To open the originally selected inventory row in unchanged attached source, return to Database connections and click Source. For another traced node, follow that node first and use its own Source command.

Both directions includes serialized controls and fields, data-source and dataset references, rule context, procedure reads and writes, operations and file-output candidates. This is a source relationship neighborhood. A procedure that reads one variable and writes another does not establish a value transfer between them. Dependencies are explicitly labeled separately from context. Choose Inputs for upstream dependencies or Destinations for downstream dependencies, then click Trace lineage again. These directions omit broad procedure read/write/handler context while retaining captured binding and rule context.

Click Lineage diagram to open Graphs with Data lineage selected. Orange dashed arrows show qualified source relationships. The HTML diagram labels arrows and lists unresolved breaks below it. Select a component node to open Database connections or a declaration/output node to open Explain. A native graph remains available when the embedded browser is unavailable. Exact node identity is retained and grouping is disabled. To return to the textual route, choose Database connections in the page selector, select the intended starting row and click Trace lineage.

A file destination node identifies a captured writer and whether the destination expression was literal or dynamic. Follow it to Explain and inspect captured source for the expression. Projected lineage omits literal contents and configuration values. No file is produced and no SQL is executed by the trace. Unknown handlers, absent dataset references, unavailable semantic targets, connection boundaries and omitted routes remain explicit. A final captured endpoint does not establish successful delivery, current values or runtime execution. Limits are 200 returned nodes, 800 edges and 12 hops; index limits are also reported. Full directions and a controlled practice project are in docs/guides/Data_Lineage_Run_and_Test.txt.

Inspect migration and schema-version history

Start bin/Win32/Debug/DelphiProjectHandoff.exe and attach the application you intend to document, or open its captured database. In the upper-right page selector choose Database schema. In the leftmost kind selector choose migration. Keep the state selector at All to include readable script candidates and migration gaps. Enter part of a script filename or an explicit user_version assignment in the search box. Select a row.

A migration changes database structure or data between application versions. DPH shows each referenced captured SQL script as a candidate. The evidence memo begins with a filename order candidate, explicit target-version assignments, verification requests, applied status and source fingerprint. Leading numeric filename prefixes provide intended-order candidates within each directory. Equivalent numbers such as 001 and 1 produce an explicit conflict. File numbers and captured loader enumeration do not establish the order used by a migration runner.

SQLite user_version assignments with a supported nonnegative integer preserve their schema qualifier and statement line. Multiple assignments remain visible; DPH does not select a deployed winner. A query of user_version and SQLite schema_version are not promoted to an application target version. Other migration-framework version metadata and dynamic expressions remain unknown. Captured integrity_check, quick_check and foreign_key_check requests identify potential verification work; DPH does not run them and does not report a successful result.

Drag the command bar scrollbar right to reveal the relationship selector and Follow relationship. Choose a captured schema evidence link and click Follow relationship to inspect the script declaration or gap. To read a script, select its migration row and click Open source. Source opens only when the attached bytes still match the capture. A migration gap cites the Pascal loader of an unavailable script; no missing SQL is reconstructed. Clear filters and use First page or Next page for larger inventories.

Applied state remains unknown because source text is not a deployment ledger. Ask the maintainer to supply the intended migration framework, ordering rules, expected schema versions and recorded verification results. Save attributed explanations through Knowledge and review them against evidence. Full commands and a controlled practice project are in docs/guides/Schema_History_Run_and_Test.txt.

Inspect import and export file formats

Start bin/Win32/Debug/DelphiProjectHandoff.exe. Attach the application you intend to document or open its captured database. Choose Import/export formats in the upper-right page selector. A file-format contract explains how another program can produce or consume a file: its encoding, structure, required fields, compatibility and error handling.

Use the kind selector to choose import, export or captured resource. Enter a format, routine or encoding in the search field. Select a row and read Format evidence and saved contract below the list. First page and Next page navigate inventories over 250 rows. Scroll the command bar horizontally when the window is narrow. Open source opens unchanged attached evidence; Explain routine selects the captured enclosing routine.

Captured JSON, XML and INI resources expose bounded member structure with scalar values omitted. This is observed structure, not a formal validation schema. Transfer API spellings identify candidates and literal filename extensions; explicit TEncoding arguments remain source candidates. Captured source decoding is separate from a transfer encoding contract. Dynamic paths, defaults, missing structure and unsupported formats remain unknown.

Click Record/open contract. An existing contract opens in Knowledge; otherwise DPH prepares a draft with fields for format/version, purpose, encoding and BOM, schema, delimiters and escaping, producer, consumer, validation, compatibility and verification. Complete each field or explicitly record Unknown and its responsible maintainer. Enter the named author, click Save and review separately against evidence. Preparing the draft does not save or verify it. Clear an unsaved Knowledge draft before opening a different contract.

Contracts retain an exact captured source identity. Reassessment with changed source creates a different contract identity; previous records remain available in Knowledge. DPH does not execute a transfer or infer successful compatibility. Detailed commands and controlled practice instructions are in docs/guides/File_Formats_Run_and_Test.txt.

Record target data retention and archival policies

Start bin/Win32/Debug/DelphiProjectHandoff.exe and attach the application you want to document, or open its saved database. Choose Target data retention in the upper-right page selector. This page records the selected application's data obligations and archival procedures. DPH database snapshot retention remains on Database tools and is a separate feature.

Click New policy. DPH opens an unsaved Knowledge draft. Replace the default title with a descriptive data scope before saving; saved titles remain stable. Enter the named contributor and complete the body below its first heading. The prompts cover data categories and tables/files, policy basis, retention period and clock-start event, expiry calculation, archive trigger and schedule, archive destination and permissions, disposal, legal holds, ownership, retrieval, failure monitoring, verification evidence and remaining unknowns. Record Unknown and its responsible maintainer when a fact is unavailable.

To link an implementing routine, choose Knowledge links in the page selector. Select entity in the target-kind selector, enter the routine name, and click Find targets. Select the semantic_routine result, choose depends_on, then Add link. To cite an operating procedure saved in Knowledge, choose section, search its title and Add link with cites. Return to Knowledge and click Save. Links are explicit captured identities or

exact section revisions; a written job name alone does not resolve an implementation link. Link search returns up to 200 candidates, so narrow the filter when necessary.

Return to Target data retention. Filter by review state or search policy text, contributor, scope or job. Select a saved policy to read its original contributor, current revision author, reviewer and explicit link count. First page and Next page cover registers larger than 250 policies. Scroll the toolbar sideways in a narrow window. Open policy loads the saved Knowledge record for editing and named review. Implementation links opens its saved links; select a link and Open saved target to inspect its captured routine or exact procedure revision.

Save creates a pending review revision. Review separately against policy evidence; editing a reviewed policy reopens review and retains earlier revisions. Save or discard an existing Knowledge draft before opening another policy. DPH does not execute archival, deletion, SQL or target jobs. Human review does not establish legal compliance or observed success; Unknown fields remain unresolved. Detailed run, practice and test commands are in docs/guides/Data_Retention_Run_and_Test.txt.

Record target data quality rules and check reports

Choose Target data quality in the upper-right page selector after attaching a source project or opening its database. New check prepares an unsaved Knowledge draft. Enter a descriptive title and named author. Preserve the first heading and record the check purpose and version, tables/fields/files and environment, quality dimension, measurable threshold and units, prerequisites and sampling, implementing procedure and schedule, responsible owner, failure severity and remediation, observed results, evidence and remaining unknowns. Unknown is an unresolved answer and should name its responsible maintainer.

Use Knowledge links to attach captured semantic routines, constraints or exact procedure-section revisions. Choose entity, evidence or section, narrow the search, Find targets, select a result, choose depends_on or cites, and Add link. Return to Knowledge and Save. A typed routine name in prose is not a resolved link. Saving leaves needs_review; named review is separate. Open check returns to the saved record for editing, history and review. Implementation links opens its exact links. Search and review-state filters narrow the register; Next page covers lists exceeding 250 rows.

To preserve an existing check report, enter the report contributor name and the check/environment/measurement context on Target data quality, then click Import check report. Select an explicitly saved UTF-8 .txt, .log or .json extract no larger than 32 KiB. Import immediately saves a new pending-review quality record containing the literal text, contributor, context, import UTC, captured generation and original byte SHA256. No file commands or markup execute. The imported text may contain sensitive data, so prepare an appropriate focused extract first. Report measurements, timestamps and runtime origin remain contributor claims.

To connect a report with its rule, Open check on the rule, choose Implementation links or Knowledge links, select section and search Imported check report plus its descriptive context. Choose the exact report revision, cites and Add link, then return to Knowledge and Save. Earlier linked revisions remain exact when a report is edited. Read every report and procedure against its source; human review does not establish live data quality. Save or discard a current draft before opening another record or importing a report. DPH database integrity checks are a separate feature.

In a narrow window scroll each command bar horizontally to reach all commands and report fields; the labeled evidence memo scrolls independently. Detailed run, practice, import and test directions are in docs/guides/Data_Quality_Run_and_Test.txt. DPH does not run target SQL, checks or applications. Captured

schema constraints and validation routines are declarations and candidates rather than observed enforcement.

Compare screen history and visual changes

Open the intended saved project, then choose Screen history in the upper-right page selector. Import actual PNG evidence on Screenshots first: enter a named contributor and a caption describing the screen, application build and conditions, then click Import an existing PNG screenshot. Each image belongs to the generation displayed at import. Import the earlier image while its matching generation is current; after reassessment import the later image. A caption does not verify an image against source.

Select the Before and After screenshot records by artifact ID, generation and caption. The newest 200 imports are available. Choose Imported images and click Compare. Scroll the command bar horizontally to reach hidden commands. Scroll the page vertically to see the previews, change list and evidence memo. Swap sides exchanges both images and form paths; click Compare again. Changing a selection clears the previous result and discards a late worker response.

The pixel mask uses top-left coordinates and compares exact decoded RGB after transparency is composited on white. Red marks changed overlapping pixels; magenta marks an extent present on only one side; white means unchanged or outside both. The evidence memo reports counts, inclusive changed bounds, original dimensions, contributors, import times, generation IDs and SHA256 fingerprints. Images are fitted only for display; comparison does not resize, align, normalize color profiles or apply a tolerance. Animation, theme, DPI and content changes can affect pixels without proving a defect or its cause.

For captured layouts enter both relative form paths, for example forms/Main.dfm, before clicking Compare. Choose Captured layouts to see blue Before geometry, orange After geometry and their overlay. Both retained captures must use the same project and analysis selection with compatible parser contracts. These drawings are approximate reconstructions, not runtime screenshots. They omit style/custom-paint behavior and show at most 300 component rectangles per pane. Missing property values remain unknown, default or inherited.

Filter the layout change list by component path or serialized values. First page and Next page show 250 rows at a time. Select a row to read added, removed or changed classes/properties and their exact before/after evidence. Renames appear as removal plus addition. Missing historical captures or form paths produce an explicit layout gap while keeping the valid image comparison. Keep necessary releases through Database tools before retention removes a capture. Full executable paths, practice steps and test commands are in docs/guides/Screen_History_Run_and_Test.txt.

Record user actions and business purposes

Open the intended project and choose Action purposes in the upper-right page selector. Each captured UI object is a candidate. Select a row to see its caption or event binding, exact source identity and saved review state. A caption or handler name does not establish a business purpose. Not recorded means a knowledgeable contributor has not supplied it.

Scroll the command bar horizontally to reach Edit purpose. Enter a named contributor and the action name, business purpose, numbered user steps, expected outcome and evidence. Also record the screen and user role, trigger, prerequisites and permissions, and failures or recovery. Unknown answers must remain explicit;

never enter credentials. The three target identity fields are read only. Duplicate captured paths disable mapping until the ambiguity is resolved.

Click Apply to Knowledge draft, then click Save on Knowledge. Apply alone does not save. Add exact evidence using Knowledge links if needed. Inspect the immutable history, enter a named reviewer and click Verify review only after checking the claims and current evidence. Editing a purpose reopens review. Saving or approving prose does not prove a runtime outcome.

Choose New task for a project procedure with no UI mapping, then complete the same fields, Apply and Save. Use the review-state selector or text filter to find recorded and unrecorded work. First page and Next page show 250 candidates at a time. Open in Knowledge exposes history; Component source and Explain object follow captured evidence when available. Source navigation requires attached unchanged text. Missing saved targets remain historical or unresolved.

Saved purpose records participate in project search, canonical handoffs, exported HTML and the user manual. Review the generation and state before sharing. Full executable paths, a controlled practice example and developer test commands are in docs/guides/Action_Purposes_Run_and_Test.txt.

Write attributed knowledge

Use the Knowledge tab for notes, business rules, engineering rules, user help, maintenance, build and deployment details, troubleshooting, orientation, decisions and unknowns. Select the relevant Explorer object before starting New to link the record to that object. With no selected object, a new record applies to the project.

Enter a stable title, choose its kind, supply a named author and write the body. Save creates a new immutable revision pending human review. Titles and kinds remain stable after creation; update the body, or create and retire a replacement. Selecting a saved row displays its text, links, current state and complete authored and review history.

Unsaved text survives a background refresh. Save or explicitly Discard edit before switching projects, selecting another record, starting another record or closing. DPH will not silently approve an old saved revision while different unsaved text is visible.

Link authored sections to exact evidence

In Knowledge, select the document kind and intended audience before saving. The section ID and revision label identify the stable record and immutable revision. Editing creates a new pending revision; it does not overwrite a previously reviewed statement.

Choose Knowledge links. Select entity, relationship, evidence, screenshot or section, enter a target filter and refresh candidates. Select the intended target and the role explains, cites, depends_on or illustrates. Add the link, then return to Knowledge and click Save. Removing a link is also an unsaved edit until Save. A section can contain up to 64 explicit links.

Open a saved target to inspect its object explanation, normalized relationship or evidence details, imported PNG, or exact historical section body. A section link pins a particular revision. If its target is updated, explicitly relink and save after reviewing the new content. DPH does not silently transfer your approval to a newer target.

Export applies audience and source scope. Out-of-scope links and backlinks are omitted with counts. An audience filter is a document organization choice; it is not a confidentiality boundary. Use explicit exclusions and inspect the resulting bundle before disclosure.

Review and retire knowledge

Enter the named reviewer and select a saved revision. Verify review records your explicit approval of that text against the captured generation. Reject records a rejected explanation. Retire preserves an obsolete record and its history while resolving its pending task. The Pending reviews only filter shows records that still need attention.

Source or configuration changes invalidate human review separately from analysis. A successful rescan preserves authored text but cannot approve it. When a linked method disappears or is renamed, DPH keeps the old link as historical evidence and refuses to transfer verified knowledge automatically. Investigate continuity and create the appropriate reviewed replacement.

Attribution is a local named-author record, not an enterprise authentication system. A verified human explanation records the reviewer decision; it does not establish unwritten historical business intent automatically. Detached review requires source attachment first.

Review changed knowledge against evidence

Choose Review tasks and refresh. Select a task to read its original trigger, prior authored text and current revision. Compare the BEFORE and AFTER panes for source presence, signature, type, hash and configuration. Drag their divider to resize them. An unavailable baseline is explicitly missing, not reconstructed from guesses.

Select an affected object and inspect Change impact or open its knowledge record. Save an authored correction before returning to task review. Enter the named reviewer and choose Verify selected only after checking the exact current content. Choose Retire selected for an obsolete section whose history must be retained.

Review requires attached current-source verification and the expected revision. A changed configuration, source hash, absent entity or obsolete rich-link target can reject approval. Resolve the underlying mismatch, refresh, and review again. Review changes the authored record state; it does not make a partial analyzer capture complete.

Answer focused author questions

Choose Author questions. DPH derives bounded questions from uncertain relationships, diagnostics, asynchronous work, SQL and output contracts, and missing reviewed user or build procedures. Select a question and inspect its evidence before contacting its author. Up to 100 questions are shown, with at most 20 per category.

Choose Record author answer. In Knowledge, enter the named author and their actual response after the Answer marker, then Save. An empty answer is rejected. The captured question remains associated with that record. Saving leaves it pending review; only named verification with no pending task hides that exact question.

Changed evidence can produce another question or reopen review. Similar wording does not establish identical evidence. Do not invent business rationale, runtime outcomes or operating procedures just to clear the list.

Preserve saved runtime observations

Select an optional object link in Explorer, then choose Runtime observations. Enter Contributor and Run, environment and outcome description. Include the actual application build, run date, platform, action, expected result and observed result. Click Select saved log and import, then choose an existing UTF-8 .txt, .log or .json extract up to 256 KiB.

Select the record to read its attribution, captured generation, linked source SHA256, original-file SHA256 and saved text. The text is displayed literally. File order and timestamps remain contributor claims; DPH did not run the application or infer causal order. Invalid UTF-8, empty extracts and damaged managed files are rejected.

Use Explain linked current object to investigate a current-generation link. Historical and unlinked records retain their text without assuming a current identity. After reopening the database, choose Refresh saved records. The newest 50 are displayed. Managed portable backups include observations; publication does not automatically share log contents.

Retain captures and review rename continuity

Choose Database tools. Preview the retention policy before applying the desired recent-capture count. Pin an important generation with a release label and named reviewer before pruning disposable cached views. Baseline, current head, recent captures and generations referenced by review or contributed evidence are protected.

Retention removes eligible derived workspace files; immutable source facts and authored history remain. It is not a command to erase the project database. Keep a recoverable copy using the explicit backup workflow before a major catalog or storage operation when needed.

If a linked object disappeared after a rename, open the continuity workflow, filter proposals and compare old and new identities. Enter a named reviewer before approving a justified mapping. Approval preserves historical links, appends an authored revision and requests review. It does not prove semantic equivalence from name similarity.

Back up and restore a complete database

Choose Database tools and click Check integrity. Read the result before relying on the database. For an explicit portable backup, choose a new .dphdb destination outside the analyzed source. The command saves a consistent SQLite copy, referenced managed views, screenshots and observations, and a SHA256 backup manifest.

Keep the backup database and its adjacent .views folder together. Choose Restore to new file and provide a fresh destination. DPH verifies filenames, manifest hashes, database integrity and managed artifacts, and refuses to overwrite an existing database. Open the restored copy and confirm the intended project, generation and authored history.

Development backups have resumed on H:, the owner-approved USB drive. Allow at least 60 seconds for the drive to wake before assessing availability. The pre-change backup is separate from the application database backup workflow. Git and both GitHub repositories also preserve validated application changes.

Publish manuals and handoffs

Open Publish. Choose Full project or Selected object and context. The task choice requires a selected canonical object and includes its incident technical context rather than a blind truncation of a full export. Enter explicit relative file or folder exclusions separated by semicolons. External source roots and sensitive filename categories are excluded by default.

Choose a new export folder name outside the source tree. DPH refuses an existing destination to avoid overwriting unrelated files. It creates offline index.html, local assets, canonical.json, editable engineering and user guides, technical JSONL indexes, evidence references, coverage and a checksum manifest. Generated AGENTS.md and redacted source browsing are separate unchecked options described below. Original source bytes are never copied.

Preview offline HTML opens the approved local index and generated hash-named object explanation pages through Edge. External addresses and arbitrary file navigation are blocked. Offline search and internal links work without a translation service. Atlas uses independently vendored DAT offline language packs; no complete DPH language catalog is supplied. The GTranslate region is disabled in the offline edition, and the language-label helper is local. No external translation or AI service receives source through this feature.

Run tools/export-pdf.ps1 with the BundleDirectory parameter for optional PDF companions. It uses LibreOffice and the approved Python 3.13 with pypdf, populates cached TOC page numbers while retaining changeable fields, and refreshes bundle checksums. Read coverage and known unknowns before treating an export as a final operating manual. Automatic documentation cannot invent missing end-user procedures.

Translate and review saved sections

Choose Translations in the upper-right page selector. First select a saved Knowledge revision. Enter the source locale and requested target locale, for example en and fr. Click the language-pack selection command and select an explicitly supplied DAT JSON pack. DPH requires its applicationId to be DelphiProjectHandoff. A matching base pack may serve a regional request; the requested and actual locales are both displayed.

Compare the labeled source and localized panes. Source phrases absent from the pack remain unchanged with a fallback count. Code, SQL, paths, hashes and protected identifiers retain their original characters. Fence unusual technical material in backticks or code blocks before translation. A terminology JSON object supplies exact prose replacements; changing it creates a newly pending derived result.

Review the source section against current attached evidence and resolve its pending tasks before approving a translation. Enter the translation reviewer name and approve only complete, accurate wording. A fallback cannot be approved. Editing source prose or invalidating its source evidence invalidates approval. Translation review does not verify the underlying business explanation.

For a small practice only, save the exact body Run conversion and use samples\translation\DPH.fr.sample.json. The result is Exécuter la conversion. This pack has only three unreviewed phrases and is not a complete language catalog. Existing entries survive closing and reopening the database.

Publish localized text and source browsing

Choose Publish. Leave Localized prose locale blank for normal publication, or enter the exact target locale used for reviewed saved-section translations. Export to a new folder. Open index.html and the localized authored-sections link. Localized_Sections.docx is the editable companion; the optional PDF command converts it when present. Inspect translated and fallback counts before sharing.

Only reviewed exact-revision authored section bodies are localized. Titles, generated guidance and technical indexes remain in their source language. Missing, stale, unreviewed or redacted sections retain explicitly labeled source prose. Each localized entry carries its source revision, source generation, requested and actual locales, reviewer and review date.

Include redacted source browsing pages is unchecked by default. Check it only when you intend to share captured code. DPH verifies available eligible files against their captured byte hashes and code page; changed or missing source rejects export. Task exports include cited noncontainer line excerpts, mark omitted lines and retain original line anchors. Shared source lines can contain surrounding code.

Open Browse captured source files and omissions from index.html, or use an object explanation's captured-source link. Binary or undecodable material, private-key material, files over 4 MiB and files over 20000 lines are omitted with reasons. Source browsing derives redacted HTML; it does not copy original source bytes. Review exclusions and output because redaction cannot recognize every sensitive format.

Include generated agent boundaries is also unchecked by default. Checking it writes DPH-generated instructions for that captured scope; it never copies the owner's AGENTS.md or grants authority to modify or execute a selected project. The bundle includes project summary, architecture, build and test, known risks and decisions narratives with explicit unknowns.

Validate a bundle in the application

Choose Validate bundle. An open database is optional. Paste a trusted 64-character captured input fingerprint, or open the trusted database first and click Use opened capture fingerprint. A fingerprint read only from an untrusted bundle cannot establish its origin.

Click Select manifest and validate bundle and select manifest.json at the bundle root. Wait for the background result. Read PASS or FAILED and every reported qualification. DPH checks names, hashes, canonical provenance, technical indexes, ranges, source browsing and localization metadata. Integrity does not prove author identity, complete analysis or reviewed operating procedures.

Historical or partial bundles are rejected by default. For deliberate qualified inspection, check Allow historical/partial inspection and select the manifest again. A qualified pass remains historical or incomplete; it cannot approve missing knowledge or current-source behavior.

Validate and preserve exports

Run DPH.ValidateHandoff.exe with the bundle directory and a separately retained trusted input fingerprint. It rejects stale exports by default. Add --allow-stale only to inspect the integrity of a historical or partial bundle; the flag does not establish current or complete source evidence.

Validate artifact filenames and SHA256 values against validation/manifest-sha256.json. The manifest records generation, input fingerprint, template version, scope, stale state and canonical hash. The validator also

checks symbol, relationship, control-flow and evidence references and source ranges. A changed expected fingerprint rejects a stale bundle.

Hashes detect changed artifacts relative to their checksum index. They are not a digital signature proving the identity of an untrusted bundle author. Keep a trusted copy of the source fingerprint and export identity. Redaction recognizes common credential assignments, access keys, tokens and private keys; explicit exclusions and a disclosure review remain necessary for sensitive projects.

Keep the dphdb database together with its managed dphdb.views folder. The database is the authority for authored knowledge; generated manuals and diagrams are derived outputs. At the owner's direction, H: development backups are enabled. Allow at least 60 seconds for this USB drive to wake before assessing availability. Validated changes are still committed and pushed to both configured GitHub repositories.

Troubleshoot common states

PARTIAL analysis: read diagnostics and source availability. Unavailable framework units and unsupported syntax remain visible gaps. Do not clear the label by treating unresolved facts as verified. Source changed during analysis: allow reassessment or invoke Full reassessment after the source settles.

Cached artifact missing or damaged: retain the database and its previous backups, then attach the correct source to rebuild a derived workspace. Different source root: DPH rejects cross-project reuse; relocation must preserve and verify project identity rather than attach an unrelated project. Knowledge changed since loading: refresh and review the current immutable revision before saving.

Embedded preview unavailable: inspect index.html in the exported folder and check the deployed Microsoft loader and WebView2 Runtime. No named author or reviewer: enter an attributable name. Closing waits for background work: cancellation is checked at safe stage boundaries; a long parser stage does not execute the analyzed program.

Practice a maintenance handoff

Locate startup from the selected project inventory. Find the main form and a visible action, follow its serialized event to the handler, inspect callbacks and downstream calls, and distinguish certain edges from inferred ones. Locate a dry-run branch or another configuration-dependent path, then identify the model used to produce a report.

Record what the evidence establishes and what remains unknown. Add an attributed author answer for missing business rationale, reassess a controlled source change, and verify that the explanation remains pending review. Export a focused task bundle and validate its hashes. This scripted practice supports onboarding; it does not replace the required independent first-time programmer acceptance exercise.

Understand implementation status

The supplied Phase 1 inventory contains 533 numbered requirements plus presentation rules. All 510 formerly unassessed requirements were reconciled in 21 audit batches. No unassessed items remain. The traceability ledger and implementation backlog distinguish implemented features, attributed manual capture, partial behavior and missing features. After the glossary, keyboard, command inventory, dynamic control, layout rule and component dependency implementations, 318 requirement records remain incomplete; overlapping records can share an implementation task. Earlier WP acceptance and 0.3.0 packages do not establish completion of the larger inventory.

Exact launch and test commands are maintained in docs\guides\Run_And_Test.txt and Business_Glossary_Run_and_Test.txt. The tools\build-phase1.cmd suite checks structured profiles, source-quality findings, captured revision comparison and project catalogs; tools\build-glossary.cmd checks the glossary. A PASS applies only to the stated behavior and inputs and does not complete unrelated requirements.

Source and revision evidence

The following source files and resources define this guide. SHA256 values identify the inspected implementation. Rebuild the guides after validated changes and retain the associated acceptance evidence.

DPH.Analysis.Async.pas
`876a836f25b3d323ac4d6f635701fb4e8e97955fdff39c964368aa62f1a4c153`
DPH.Analysis.Explanations.pas
`43244ca3544edffabbf8a52af92a5f8081794894db0fd88fdd67fd1dbab560fb`
DPH.Analysis.Flow.pas
`ee7610544411a64da399659f2af11e84408d285837a45205d610d2c37af8ef57`
DPH.Analysis.Model.pas
`93d7e969508ff4862a98f7e4d7b6cb690cf741792d5a0e4dbac9f012bc8c5035`
DPH.Analysis.Persistence.pas
`08609c3df54250a8f6712e22e991f7ad8d444352e7103ba0959d2feba4a2b3c7`
DPH.Analysis.Quality.pas
`50196d33a8d15cfeb52528dde93a2bfdc22b153e2533830b08e3ac097e14fd82`
DPH.Analysis.SQL.pas
`68ea587d47e2c7ebf660f85eed6b605f6076480ead3a461742b8290fec7a130e`
DPH.Analysis.Symbols.pas
`47c187a32ad1a040760bd9459e843d9c6fa7ced7b4ea5843280d2680127cc047`
DPH.Knowledge.ActionPurpose.pas
`bd548e1d5bbb886d3d3f2b5ba59d1cd86144a92877eba20596f956582740573f`
DPH.Knowledge.ComponentDependency.pas
`56bc1021fffa88f0a383a2a159b4c74328d735597f72c3ec279ef42ad2a2ccbe`
DPH.Knowledge.Glossary.pas
`9c50a06ccf405f786526e206e5e84ea956604eeb42e1a84b9f91c31cced70dcb`
DPH.Knowledge.ProjectProfile.pas
`e901a58b02c2aafe2cc21df4da7dced2807a9cc8840c325c72d9db64cedc44bd`
DPH.Parse.Forms.pas
`3cf7d49afd859e2d2749cb30e5fdd76120eaf05ef5223e8fb8bf6eb1a7fcd66c`
DPH.Parse.Lexer.pas
`6ce5bbc66f8170b17eaeda536b8e500ca9992131477c09bc8e1a33bb6dd1c7f0`
DPH.Parse.Model.pas
`bdecd758070044593a6a19bfd53b35f9a23286e2542194313222cdfee51d6557`
DPH.Parse.Pascal.pas
`016adbe9ba24b90d457455722782cdbbc657233bd1a07ba1bcdcd77f3340587e`
DPH.Parse.Preprocessor.pas
`34b7513019a41bc9911babfda72041addd563959ee54c54d0759b8cc9fa7fefc`
DPH.Parse.Resources.pas
`f63153edba8656b315956fdd8ccf52e45b1446d8192f9b84c96c153bf676258f`
DPH.Project.Analysis.pas
`5af3808df19a7c403b456342e45b9e2c8c63195368847b151f3c57ed461b5c2a`
DPH.Project.Inventory.pas
`061736891f0fab486996ff5d8132af1cf5e25a2f93a74fb3e03ad8fa04e2c266`
DPH.Project.Source.pas
`8efa444743f0e90831631df963f26a2542561efab63b26f1c5b41fff6b96994e`
DPH.Publish.BrowserPolicy.pas
`74d2f36c879f1b7b55133241c3e1abc28ce113c4d9f581a5123be83f94d705ce`

DPH.Publish.Export.pas
0a36aa00c6c9591521d53977e2a4985b4cb390dfa79c7dcbc337c74ff306c39e
DPH.Publish.Model.pas
532727b11c1245adceaab9e43a6f16d4bce573cf1f31c148978a78457462039f
DPH.Publish.Translation.pas
978be4250a2332736f73cd4e56e85d5c2f0b0d3a6224a4db2f8f3b7b5f4cde20
DPH.Storage.Backup.pas
78e8c8de8af5120f8ce453793f947c5e2918e51dafc66f9e129c964fe28697bc
DPH.Storage.Database.pas
19a2c54d28475de14fc6419deb149cdc7df2d4503077049a542fe5ca02ca6077
DPH.Storage.Types.pas
1e83ccbe6868979ff9d763a4f2bf4d64691c2b2343de6717391f03e204d5d219
DPH.UI.ActionPurpose.pas
6016874bdc24964ce2371431d77babc68f03a50f045e8d20e222e4989bcbf79d
DPH.UI.ComponentDependency.pas
dc711088afbccd032f4511b9fe3da95b89640b505edb1868ee7b47c7e2a64682
DPH.UI.DatabaseDictionary.pas
c55bc57ac98df6caa84f29c5b775b4915c8afaf3837859f3e197f8b1fffbf90b
DPH.UI.Glossary.pas
fb15295b29a9bbb9caa05cea723e737c81b2418a271f59fb6071dbffe37f8741
DPH.UI.Main.pas
96021c11323fa9db76d9b2bb7d8ea4f0ee1699fac73f675dfecade849fd5447a
DPH.UI.ProjectProfile.pas
f3fe7ed94987e956999e7449887cd8ca259f009b3d4ac934c1e8e09b8e9fab59
DPH.Workspace.ActionPurposes.pas
e720b935f3d9f38f07348bead5f21c9c9e3ec89ad102f4dae0c6e902c0a101e4
DPH.Workspace.Catalog.pas
5fed34d1919dfd5b34216be36f92fc1764847a65d62112613b2a0e1d44a05940
DPH.Workspace.Commands.pas
9d737351356cf30b1f5bd6d2dee5c514bcafe16c6fcafe24be82b2f06c1a3d28
DPH.Workspace.Comparison.pas
d4953d401b70fbe65b8b9b9fe387431cb7e2c84afc5dab5c9170deafe9ceacce
DPH.Workspace.ComponentDependencies.pas
7d57eb7884d6e521414379a24ce9e64aa2816beca6de650b86828a16fc33af2b
DPH.Workspace.DatabaseConnections.pas
37fecd28b258661a57eb1ff955a2bf1f22757508b3925b98daedc0d6b789e1ce
DPH.Workspace.DatabaseSchema.pas
988548013d7465e7a95b4b1729c0fec5938f5d866afda932bfeab22639160bda
DPH.Workspace.DynamicControls.pas
abd5350eab05058b885b95e0c947a1badd54eba33877cbe9a448cac194db25ed
DPH.Workspace.FormInheritance.pas
b771383670720f9503454dc17f7d5e72ddd986a642fc526bc6aeeab6122129f8
DPH.Workspace.Graphs.pas
6fd8b84644df65a0746aaad3598dad9406fa2d55e637e1e30f2175cefd1be911
DPH.Workspace.Keyboard.pas
6e854d724e8360e3d5015d63077744636cf88f33eed5c9fcff175ef01e0f9923
DPH.Workspace.LayoutRules.pas
02dcd1a040fc209a557281fe8150bd40a59291a18a08fa48d5e07065dbac1c49
DPH.Workspace.Model.pas
1e5c536c3fb44fd001a56177e714f5fd08b62280cbb814973c76005ad0d88c0d
DPH.Workspace.Screen.pas
68305e6ec36664e76d05b8c9af21416dd51a32116d47ea55156977b4c10855de
DPH.Workspace.ScreenHistory.pas
bd05a76dcae7b4c872345021a56eab0b8b0108d954c1a5c35686321852082161
DPH.Workspace.Worker.pas
d6ac3ffff0c50dbcbb98f0813db1ea47769f0fa8e1f2e9418381c6735cc09953
DPH.UI.Main.fmx
e3659f88aee88537216fa8f7c6d1250d08954d2465ad2d6cde4e8b1960ef9f00

DPH.UI.ProjectProfile.fmx
62e7fef97972fdb29f1a878e70a99d8284b6f7d8cfbddb9f625bb535728c2142
DPH.UI.Glossary.fmx
3334c41ac643daa3596f1a5c9cb8fb65c3447daf77cf00006d6a0ca7a9d517d5
DPH.UI.ComponentDependency.fmx
3b604fcf033e37ebda4aa6968e1a9e6ffe11bf40416fac5fbdd39fa2599d7862
DPH.UI.ActionPurpose.fmx
9eb57da0310277432db54c9a80d8d03be09aa6163acd8895f04fa21f176f8e15
DPH.UI.DatabaseDictionary.fmx
5b40f12073d093c4e0b7f708440dd125662dc8e76a0b3c3bf30a4c749c69484e
DelphiProjectHandoff.dpr
2b922d5561c2ab3e2b65590583f646e29fc12c8b5c36a08fe401a6ebed90d3f9
DelphiProjectHandoff.dproj
fc131b87384bc80f6d391db9c015b7aaa41c098ff8b96d53abfcdb67ced18e8d